Tier-1 Publisher License & Audit-Risk Management (Oracle / IBM / SAP)

Implementation Solution Guide

Version 1.0.0 · 2026-06-29

Contents

Executive Summary — Tier-1 Publisher License & Audit-Risk Management (Oracle / IBM / SAP) 4

What you get . . . 4

What this delivers . . . 4

The deployment model . . . 4

Outcome . . . 4

1. Solution Overview — Tier-1 Publisher License & Audit-Risk Management 5

What "tier-1 publisher" means here . . . 5

The metrics, at a correct but general level . . . 5

How the repeatable way works . . . 5

What you can control . . . 6

Why this matters (the why lives here only) . . . 6

Limitations (be honest with buyers) . . . 6

The standard deployment model . . . 6

2. Requirements Guide — Tier-1 Publisher License & Audit-Risk Management 7

2.1 Per-publisher contracts and entitlement records . . . 7

2.2 Metric definitions for each product in scope . . . 7

2.3 Deployment / installation inventory for tier-1 products . . . 7

2.4 Host and virtualization topology . . . 7

2.5 Sub-capacity / lower-cost-counting prerequisites (where claimed) . . . 8

2.6 Reporting toolchain for position and exposure . . . 8

2.7 Owner sign-off and the exposure register . . . 8

2.8 Pilot publisher scope . . . 8

Checklist . . . 8

3. Implementation Guide — Tier-1 Publisher License & Audit-Risk Management 9

Step 1 — Choose the pilot publisher and build the scope . . . 9

Step 2 — Gather contracts and define the metric per product . . . 9

Step 3 — Discover deployment . . . 9

Step 4 — Map deployment to the licensing metric . . . 10

Step 5 — Calculate position and exposure . . . 10

Step 6 — Close gaps and set controls . . . 10

Step 7 — Validate the pilot publisher . . . 10

Step 8 — Prepare the audit evidence pack . . . 11

Step 9 — Expand to the remaining publishers . . . 11

5. Testing Guide — Tier-1 Publisher License & Audit-Risk Management 11

Test 1 — Scope is complete . . . 11
Test 2 — Every product has a defined metric . . . 11
Test 3 — Deployment maps cleanly to the metric . . . 11
Test 4 — Position and exposure are calculated . . . 11
Test 5 — Every exposure item has an owner and disposition . . . 12
Test 6 — Remediation proposes, does not change . . . 12
Test 7 — Audit evidence pack is assembled . . . 12
Pass/fail gate . . . 12
6. Troubleshooting Guide — Tier-1 Publisher License & Audit-Risk Management 12
A. Deployment will not map to a metric . . . 12
B. Position looks wrong (too good or too bad) . . . 13
C. Discovery is incomplete . . . 13
D. Options / add-ons appear unexpectedly . . . 13
E. Remediation concern . . . 13
F. Quick reference . . . 13
G. When to escalate . . . 14
7. Best Practices — Tier-1 Publisher License & Audit-Risk Management 14
8. Deliverables Summary — Tier-1 Publisher License & Audit-Risk Management 15
Documentation . . . 15
PowerShell scripts . . . 15
Artifacts produced (named, reused per publisher) . . . 16
What the buyer receives . . . 16
Outcome . . . 16
Appendix A — PowerShell Scripts 17

Executive Summary — Tier-1 Publisher License & Audit-Risk Management (Oracle / IBM / SAP)

Oracle, IBM, and SAP are a licensing category of their own. Their metrics are complex (processor cores, named users, value units, digital access), their audit programs are aggressive and well-resourced, and a single finding can turn into a seven-figure settlement. Most organizations discover their exposure only when an auditor does — by then the leverage is gone.

This package gives you a repeatable way to manage tier-1 publisher licensing and audit risk before that happens: understand each publisher's metrics, map your actual deployment to the metric that governs it, calculate a defensible licensing position, surface exposure early, close gaps on your own terms, and put controls around your highest-risk software. It is written for a SAM analyst and is designed to be run on one publisher first, then repeated across the rest.

What you get

• A plain-language `Solution Overview` of the tier-1 publisher landscape and the key metrics —

Oracle Processor and Named-User Plus, IBM PVU, SAP named-user and digital access — and why audit risk is high.

• A `Requirements Guide` with verify / implement-if-missing for every prerequisite, including the

contracts and entitlement records you must gather per publisher. Precise contract terms are deferred to your agreements and the publisher.

• An `Implementation Guide` of precise, ordered steps: per publisher, gather contracts and metrics,

map deployment to the licensing metric, calculate position and exposure, close gaps, set controls, and prepare for audit.

• Five PowerShell scripts that work on inventory exports / CSV: deployment discovery, metric-mapping

validation, a per-publisher position and exposure report, safe remediation of over-deployment, and a publisher-scope group builder.

• `Testing` and `Troubleshooting` guides with explicit pass/fail gates, plus field-proven

`Best Practices`.

What this delivers

A calculated, defensible licensing position for each tier-1 publisher — deployment mapped to the correct metric, the entitlement-versus-consumption gap quantified, exposure recorded with an owner, gaps closed or budgeted, and controls in place — so a tier-1 audit holds no surprises.

The deployment model

Assess to Configure to Pilot to Validate to Expand — proven on one publisher before you expand to the rest.

Outcome

The highest-risk publishers under control: Oracle, IBM, and SAP positions calculated against their own metrics, exposure identified before an auditor finds it, and controls that keep the position defensible cycle after cycle.

1. Solution Overview — Tier-1 Publisher License & Audit-Risk Management

What "tier-1 publisher" means here

Oracle, IBM, and SAP are the publishers most likely to audit you and most expensive to get wrong. Their licensing differs from ordinary per-device or per-user software in three ways: the metrics are indirect (you are not licensed per install — you are licensed per processor core, per named user, or per value unit), the rules for counting are detailed and publisher-specific, and the audit programs behind them are formal, scheduled, and designed to find shortfalls. This solution is the repeatable way to manage that risk: know the metric, map your deployment to it, calculate where you stand, and control the gap before an auditor does.

The metrics, at a correct but general level

Precise rates, core factors, user-type definitions, and price lists live in your own agreements and with the publisher. Treat the summaries below as the model, not the contract.

• Oracle — Processor and Named-User Plus (NUP). The Processor metric counts the processors (cores adjusted by Oracle's core factor) where a program runs or is installed and available to run; virtualization and clustering rules strongly affect what must be counted. Named-User Plus counts individuals and devices authorized to use the program, subject to per-program minimums. Options and management packs are licensed separately and are a common source of unplanned use.

• IBM — Processor Value Unit (PVU). Eligible products are licensed by the number of value units assigned to each processor core the software can run on, where a per-core value-unit rating applies. Sub-capacity (licensing only the virtual capacity used) is permitted only when the required metering tool is deployed and reporting; without it, full-capacity counting applies.

• SAP — named-user and digital (indirect) access. Named users are licensed by user type/category according to what each person is entitled to do; engine/package metrics license specific functional consumption. Digital access addresses indirect/third-party and automated consumption of SAP data (commonly measured by document volume) rather than by a human logging in.

How the repeatable way works

1. Per publisher, gather the contracts and entitlement records and confirm which metric governs each product you run (the per-publisher position worksheet).

1. Discover what is actually deployed for that publisher from your inventory exports and map each deployment to the licensing metric (deployment-to-metric mapping).

1. Calculate the licensing position: entitlements held versus consumption required by the metric.

2. Quantify the difference as exposure (a shortfall) or headroom (a surplus) and record it (the per-publisher exposure register).

1. Close gaps on your terms — reduce deployment, re-deploy to lower-cost configurations, or acquire licenses — and put controls around the highest-risk products (the tier-1 publisher controls).

1. Prepare the evidence pack so an audit is a confirmation, not a discovery.

What you can control

• Which deployments are counted, and how, by removing unused installs and unlicensed options before they are measured.

• Where high-metric software runs (for example, isolating processor-counted software so unrelated hosts are not pulled into scope).

• Whether sub-capacity / lower-cost counting is available to you, by ensuring the prerequisites for it are actually in place and reporting.

• Who owns each exposure item and the decision to remediate, budget, or accept it.

Why this matters (the why lives here only)

Tier-1 audits are not random. They are triggered by renewals, mergers, big infrastructure changes, and lapsed engagement, and they are run by teams whose job is to find the gap between what you deployed and what you licensed. The organizations that come through them cheaply are the ones that already knew their position, had already closed or budgeted the gap, and could hand the auditor a worksheet instead of root access. Calculating your own position first converts a one-sided audit into a negotiation you are prepared for — and it is the only point in the process where you still hold leverage.

Limitations (be honest with buyers)

• This is a methodology and tooling package, not legal or contractual advice. Exact metric definitions, core factors, user categories, and rates are governed by your agreements and the publisher and must be confirmed there.

• The scripts operate on inventory exports / CSV, not on vendor-proprietary measurement tools; they estimate position from the deployment data you provide and are only as good as that inventory.

• Verified metric outputs (for example, official sub-capacity reports or a publisher's own measurement scripts) remain authoritative for a formal audit; this package builds your internal, defensible position ahead of that.

The standard deployment model

> *Per publisher: gather contracts and metrics, map deployment to metric, calculate position and exposure, close gaps and set controls, prepare for audit — pilot one publisher, then expand. Model:*

Assess to Configure to Pilot to Validate to Expand.

2. Requirements Guide — Tier-1 Publisher License & Audit-Risk Management

Verify every requirement before configuring. Each entry: Description, Why, Verify, and Implement-if-missing. Category-wide prerequisites are in `global-requirements-asset-mgt.md` and are assumed here.

2.1 Per-publisher contracts and entitlement records

Description: The signed agreements and ordering documents for each tier-1 publisher in scope — master agreement, ordering documents / order forms, license metric definitions, and any amendments — plus the current entitlement quantities they grant. Why: The contract defines which metric governs each product and how it is counted; you cannot calculate a defensible position without it. Verify: For one product per publisher, you can locate the governing metric and the entitled quantity in the contract documents. Implement: Collect the agreement set from procurement / legal and record entitled quantities into the per-publisher position worksheet; where a clause is ambiguous, flag it for the publisher rather than guessing.

2.2 Metric definitions for each product in scope

Description: The specific metric (for example Oracle Processor or Named-User Plus, IBM PVU, SAP named-user type or digital access) that applies to each deployed product, with the counting rules that go with it. Why: Mapping deployment to the wrong metric produces a wrong position and false confidence. Verify: Each product line in the position worksheet has a named metric and a one-line counting rule. Implement: Confirm each metric from the contract and the publisher's current definitions; record the applicable core factor, user-type, or document-counting basis per product. Defer exact rates to the publisher.

2.3 Deployment / installation inventory for tier-1 products

Description: An export of what is actually installed and where it runs for each publisher's products — host name, the product/edition, the host's processor/core configuration, virtualization host and cluster, and (for user-based metrics) the authorized users or accounts. Why: Position is calculated by mapping real deployment to the metric; the inventory is the input. Verify: You can produce a CSV of tier-1 product installs with host, edition, and core/user data for a sample. Implement: Export from your discovery tool / CMDB / inventory (see global requirement on deployment data); where automated discovery misses a product, supplement with a manual host list. Run `TIER1LIC-Discovery.ps1` to assemble the inventory baseline.

2.4 Host and virtualization topology

Description: A record of physical hosts, cores per host, hypervisor type, clusters, and which guests can run tier-1 software — because processor- and value-unit metrics often count the host or cluster capacity, not just the guest. Why: For Oracle Processor and IBM PVU, virtualization and clustering rules can pull entire

hosts or clusters into scope; getting topology wrong is the single most common cause of a wrong position. Verify: For a sampled processor-counted product, you can state the host, cores, and cluster it belongs to. Implement: Capture topology from your virtualization platform export and join it to the deployment inventory in the deployment-to-metric mapping.

2.5 Sub-capacity / lower-cost-counting prerequisites (where claimed)

Description: The conditions a publisher requires before you may license less than full capacity — for example a required metering/measurement tool deployed and reporting, or an approved deployment configuration. Why: If you assume sub-capacity counting but the prerequisites are not met, the publisher will count full capacity and your real exposure is far larger than your worksheet shows. Verify: For any product where you claim reduced counting, the required tool is installed and producing current reports. Implement: Deploy and confirm the required measurement tooling, or recalculate that product at full capacity until the prerequisites are genuinely in place.

2.6 Reporting toolchain for position and exposure

Description: Spreadsheet / BI tooling to hold the per-publisher position worksheet and exposure register and to produce the reports (see category global requirement on the reporting toolchain). Why: Position, exposure, and gap-closure are delivered as auditable worksheets and reports. Verify: You can open the worksheet template and connect it to the inventory CSV. Implement: Stand up the worksheet/BI workbook; `TIER1LIC-Report.ps1` writes the position and exposure outputs into it.

2.7 Owner sign-off and the exposure register

Description: A named owner (SAM analyst plus a business/procurement approver) for each publisher and an agreed register where exposure items, decisions, and controls are recorded and tracked. Why: Closing a gap or accepting risk is a business decision; exposure must have an owner and an audit trail. Verify: For the pilot publisher, the register exists and has a named owner and approver. Implement: Create the per-publisher exposure register and assign owners (see category global requirement on stakeholder alignment).

2.8 Pilot publisher scope

Description: One tier-1 publisher chosen to prove the process end-to-end before the others. Why: Every solution is piloted before broad adoption to validate accuracy and avoid disruption. Verify: A pilot publisher and its data owners are identified. Implement: Choose the publisher with the clearest contracts and inventory as the first pass; use `TIER1LIC-Set-ScopeTargeting.ps1` to build the scope group.

Checklist

- Per-publisher contracts and entitlement records gathered
- Metric defined for each in-scope product
- Deployment / installation inventory exported — `TIER1LIC-Discovery.ps1`
- Host and virtualization topology captured

- Sub-capacity / lower-cost-counting prerequisites verified where claimed
- Reporting toolchain ready (position worksheet + exposure register)
- Exposure register created with named owner and approver
- Pilot publisher scope chosen — `TIER1LIC-Set-ScopeTargeting.ps1`

3. Implementation Guide — Tier-1 Publisher License & Audit-Risk Management

Follow these steps in order. Run them for one pilot publisher first, validate, then repeat for the others. The artifacts named below (the per-publisher position worksheet, the deployment-to-metric mapping, the per-publisher exposure register, and the tier-1 publisher controls) are reused verbatim for each publisher.

> *Model: Per publisher, gather contracts and metrics, map deployment to the metric, calculate position and exposure, close gaps and set controls, prepare for audit. Pilot one publisher, then expand. Assess to Configure to Pilot to Validate to Expand.*

Step 1 — Choose the pilot publisher and build the scope

1. Pick the pilot publisher (clearest contracts and inventory — see Requirements 2.8).
2. Run `TIER1LIC-Set-ScopeTargeting.ps1 -Publisher <Name>` to build the publisher-scope group from

the inventory — the list of hosts/users in scope for this publisher.

1. Record the scope as the header of the per-publisher position worksheet.

Step 2 — Gather contracts and define the metric per product

1. For the pilot publisher, collect the agreement set and entitlement quantities (Requirement 2.1).
2. For each product in scope, record in the per-publisher position worksheet: product/edition, the

governing metric (for example Oracle Processor or NUP, IBM PVU, SAP named-user type or digital access), the one-line counting rule, and the entitled quantity.

1. Where a clause is ambiguous, mark it open and raise it with the publisher rather than assuming.

Step 3 — Discover deployment

1. Run `TIER1LIC-Discovery.ps1 -InventoryCsv <path> -Publisher <Name>` to assemble the deployment

inventory: each install with host, edition, cores per host, virtualization host/cluster, and (for user metrics) authorized users.

1. Reconcile the discovered list against the scope group from Step 1; resolve any host the inventory

missed by adding it manually.

Step 4 — Map deployment to the licensing metric

1. Run `TIER1LIC-Validation.ps1 -InventoryCsv <path> -Publisher <Name>` to produce the

deployment-to-metric mapping: for each install it applies the metric's counting basis (processor cores adjusted by the recorded core factor, value units per core, named users by type, or document volume for digital access).

1. Validation flags every row that cannot be mapped (missing metric, missing core count, missing

core factor, claimed sub-capacity without the prerequisite). Resolve each flag before continuing — an unmapped row is unknown exposure.

1. For virtualized processor/value-unit products, confirm the topology decision (host, cluster, or

guest counting) is recorded per Requirement 2.4.

Step 5 — Calculate position and exposure

1. Run `TIER1LIC-Report.ps1 -InventoryCsv <path> -Publisher <Name>` to calculate, per product,

consumption required by the metric versus entitlement held.

1. The report writes the per-publisher exposure register: each product with its position (shortfall

or headroom), the exposure quantity, and a severity based on the gap.

1. Review the register with the named owner/approver (Requirement 2.7); assign each exposure item a

disposition: remediate, acquire, or accept-and-budget.

Step 6 — Close gaps and set controls

1. For over-deployment, run `TIER1LIC-Remediation.ps1 -InventoryCsv <path> -Publisher <Name>` to

produce the remediation work list (it runs in `-WhatIf` mode by default and only proposes actions — uninstall unused installs, consolidate processor-counted software onto fewer hosts, remove unlicensed options/users). Nothing is changed until an owner approves and the action is performed through your normal change process.

1. Re-run validation and the report after each approved change so the exposure register reflects the

new position.

1. Record the tier-1 publisher controls for the highest-risk products: where the software may run,

who may install it, which options are permitted, and the sub-capacity prerequisites that must stay in place. Wire these into your intake/change process so new deployments cannot silently recreate exposure.

Step 7 — Validate the pilot publisher

1. Confirm every product in the position worksheet has a mapped metric, a calculated position, and a

register entry with an owner and disposition.

1. Confirm no validation flags remain unresolved. Complete the Testing Guide pass/fail gate.

Step 8 — Prepare the audit evidence pack

1. Assemble the contracts, the position worksheet, the deployment-to-metric mapping, the exposure register with dispositions, and proof of any sub-capacity prerequisites into a single audit evidence pack per publisher.

1. Keep it current: it is what you hand an auditor instead of granting open-ended access.

Step 9 — Expand to the remaining publishers

1. Repeat Steps 1 to 8 for the next publisher, reusing the same artifacts and scripts.

2. Re-validate each publisher on a set cadence (renewal-driven at minimum) so the position stays defensible cycle after cycle.

Order of operations

```
Scope -> Contracts/metric -> Discover deployment -> Map to metric -> Position & exposure -> Close gaps & controls -> Validate -> Audit evidence pack -> Expand
```

5. Testing Guide — Tier-1 Publisher License & Audit-Risk Management

Run after the pilot publisher is processed (Implementation Step 7) and again for each publisher and each re-validation cycle.

Test 1 — Scope is complete

Do: Compare the publisher-scope group from `TIER1LIC-Set-ScopeTargeting.ps1` against the deployment inventory from `TIER1LIC-Discovery.ps1`. Expected: Every in-scope host/user for the publisher appears in the inventory; nothing in scope is missing.

Test 2 — Every product has a defined metric

Do: Open the per-publisher position worksheet. Expected: Each product line has a named metric, a one-line counting rule, and an entitled quantity — no blanks.

Test 3 — Deployment maps cleanly to the metric

Do: Run `TIER1LIC-Validation.ps1` for the publisher. Expected: Zero unresolved flags — no missing metric, missing core count/factor, or sub-capacity claimed without the prerequisite. Every install maps to a metric quantity.

Test 4 — Position and exposure are calculated

Do: Run `TIER1LIC-Report.ps1`. Expected: Each product shows consumption versus entitlement, a position (shortfall or headroom), and an exposure quantity; the per-publisher exposure register is written.

Test 5 — Every exposure item has an owner and disposition

Do: Review the exposure register with the owner/approver. Expected: Each item is assigned remediate, acquire, or accept-and-budget, with a named owner.

Test 6 — Remediation proposes, does not change

Do: Run `TIER1LIC-Remediation.ps1` (default). Expected: It runs in `-WhatIf` mode and outputs a proposed work list only; no system is modified.

Test 7 — Audit evidence pack is assembled

Do: Open the audit evidence pack for the publisher. Expected: Contracts, position worksheet, deployment-to-metric mapping, exposure register with dispositions, and sub-capacity proof (where claimed) are all present and current.

Pass/fail gate

Gate	Requirement
Pilot pass	Tests 1 to 5 pass for the pilot publisher; no unresolved validation flags
Publisher pass	Tests 1 to 7 pass for the publisher before its audit/renewal cycle

6. Troubleshooting Guide — Tier-1 Publisher License & Audit-Risk Management

Use `TIER1LIC-Discovery.ps1` to (re)assemble the inventory, `TIER1LIC-Validation.ps1` for metric-mapping flags, `TIER1LIC-Report.ps1` for position and exposure, and `TIER1LIC-Remediation.ps1` (default `-WhatIf`) to propose safe gap closure.

A. Deployment will not map to a metric

Symptom	Cause	Remediation
Rows flagged "no metric"	Product not in the position worksheet, or metric undefined	Add the product and its governing metric from the contract (Requirement 2.2); re-run validation
Rows flagged "missing cores/core factor"	Inventory lacks core count or the core factor was not recorded	Join host topology to the inventory (Requirement 2.4) and record the core factor; re-run validation

Symptom	Cause	Remediation
Named-user products show zero users	User/authorization data not in the export	Add authorized users/accounts to the inventory for that product; re-run validation

B. Position looks wrong (too good or too bad)

Symptom	Cause	Remediation
Exposure unrealistically low	Sub-capacity counting assumed without the prerequisite	Confirm the required measurement tool is deployed and reporting; if not, recalculate at full capacity (Requirement 2.5)
Exposure unrealistically high	Processor/value-unit product counted at host/cluster when guest counting applies (or vice versa)	Re-check the topology decision per Requirement 2.4 and correct the counting basis; re-run report
Entitlement looks short	Contract amendments or additional orders not loaded	Reconcile entitlements against all ordering documents/amendments; update the worksheet

C. Discovery is incomplete

- A tier-1 product is installed but missing from the inventory: confirm the discovery source covers that host; supplement with a manual host list and re-run `TIER1LIC-Discovery.ps1`. Inventory gaps are unknown exposure, not zero exposure.

D. Options / add-ons appear unexpectedly

- Oracle options/management packs or unentitled SAP user types/engines show as deployed: verify entitlement in the contract; if unentitled, add to the exposure register and treat as licensable until proven otherwise.

E. Remediation concern

- Remediation must never change a system on its own: it runs in `-WhatIf` by default and only proposes a work list. Perform approved actions through your normal change process, then re-run validation and the report so the exposure register reflects the new position.

F. Quick reference

```
TIER1LIC-Discovery.ps1   -InventoryCsv <path> -Publisher <Name>   # assemble inventory (read-only)
TIER1LIC-Validation.ps1  -InventoryCsv <path> -Publisher <Name>   # map to metric, list flags
TIER1LIC-Report.ps1      -InventoryCsv <path> -Publisher <Name>   # position + exposure register
TIER1LIC-Remediation.ps1 -InventoryCsv <path> -Publisher <Name>   # propose fixes (-WhatIf default)
```

The contract and the publisher remain authoritative for exact metric definitions, core factors, user categories, and rates — confirm there before acting on a calculated position.

G. When to escalate

If deployment maps cleanly, topology and sub-capacity prerequisites are confirmed, entitlements are fully reconciled, and the calculated position still looks implausible, escalate to the contract owner / procurement and, where the metric interpretation itself is in question, to the publisher — in writing — before an audit is opened.

7. Best Practices — Tier-1 Publisher License & Audit-Risk Management

• Pilot one publisher first. Prove the whole process — scope, metric, mapping, position, register — on the publisher with the clearest contracts before you take on the others. The artifacts are reusable, so the second publisher is far faster than the first.

• Map deployment to the metric, not to installs. The licensable unit is processor cores, value units, named users, or documents — never the raw install count. Always state the metric before you count.

• Get virtualization topology right. For Oracle Processor and IBM PVU, hosts and clusters — not just guests — can be pulled into scope. A wrong topology assumption is the most common cause of a wrong position; record host, cores, and cluster for every processor-counted product.

• Never assume sub-capacity / lower-cost counting. Claim reduced counting only where the publisher's prerequisite (for example a deployed, reporting measurement tool) is genuinely met; otherwise count full capacity. Assuming it is the most expensive mistake on the worksheet.

• Watch the options and add-ons. Oracle options and management packs, and unlicensed user types or engines, are quietly enabled and counted later. Treat every option as licensable until proven entitled.

• Calculate your position before the auditor does. The only point with leverage is before the audit. Know the gap, decide remediate / acquire / accept, and act while it is still your choice.

• Give every exposure item an owner and a disposition. A shortfall with no owner is a future settlement; record it in the exposure register and track it to closure.

• Remediate safely. Run remediation in `-WhatIf` first, get owner approval, and make changes through your normal change process — then re-validate so the register reflects reality.

• Control where high-risk software runs. Isolate processor-counted software so unrelated hosts are not dragged into scope, and gate new installs through intake so exposure cannot silently return.

• Keep the audit evidence pack current. Maintain contracts, position worksheet, mapping, register, and sub-capacity proof as a standing pack — hand an auditor a worksheet, not open access.

• Re-validate on a cadence. Renewals, mergers, and big infrastructure changes shift your position;

re-run the cycle at least at each renewal so the position stays defensible.

• Defer exact terms to the contract and publisher. Summaries guide the method; rates, core factors, user categories, and definitions are governed by your agreements — confirm there, do not assume.

8. Deliverables Summary — Tier-1 Publisher License & Audit-Risk Management

Documentation

Document	Purpose
Executive Summary	Business case and what is in the box
1. Solution Overview	Tier-1 publisher landscape, the metrics (Oracle Processor/NUP, IBM PVU, SAP named-user & digital access), why audit risk is high, and the defensible-position approach
2. Requirements Guide	Every prerequisite with Verify / Implement-if-missing
Global Requirements (Asset Mgt)	Category-wide prerequisites, published once
3. Implementation Guide	Exact, ordered steps: scope, metric, mapping, position, gap closure, controls, audit prep
5. Testing Guide	Validation procedures with pass/fail gates
6. Troubleshooting Guide	Field issues and remediation
7. Best Practices	Field-proven recommendations

PowerShell scripts

Script	What it does
`TIER1LIC-Discovery.ps1`	Reads inventory exports / CSV and assembles the deployment inventory for a publisher (host, edition, cores, virtualization host/cluster, users). Read-only
`TIER1LIC-Validation.ps1`	Maps each deployment to its licensing metric and flags rows that cannot be mapped. Read-only
`TIER1LIC-Report.ps1`	Calculates per-publisher position (consumption vs entitlement) and writes the exposure register. Read-only
`TIER1LIC-Remediation.ps1`	Proposes safe gap-closure actions for over-deployment; runs in `-WhatIf` by default and changes nothing without approval
`TIER1LIC-Set-ScopeTargeting.ps1`	Builds the publisher-scope group (hosts/users in scope) from the inventory

Artifacts produced (named, reused per publisher)

- Per-publisher position worksheet — metric, entitlement, and consumption per product.
- Deployment-to-metric mapping — each install mapped to its metric quantity.
- Per-publisher exposure register — shortfall/headroom, severity, owner, disposition.
- Tier-1 publisher controls — where high-risk software may run and who may deploy it.
- Audit evidence pack — the assembled, current proof handed to an auditor.

What the buyer receives

- This implementation guide as a Word document and PDF.
- All five PowerShell scripts in a single ZIP.
- A repeatable process: Assess to Configure to Pilot to Validate to Expand, proven on one publisher before expanding.

Outcome

The highest-risk publishers under control — Oracle, IBM, and SAP positions calculated against their own metrics, exposure identified before an auditor finds it, and controls in place so a tier-1 audit holds no surprises.

Appendix A — PowerShell Scripts

Included in the accompanying ZIP. Each script's header documents full usage.

TIER1LIC-Discovery.ps1

Purpose: Reads an installed-software / host inventory export (CSV) and produces the deployment baseline for one tier-1 publisher (Oracle / IBM / SAP): each install with host, product, edition, cores per host, virtualization host/cluster, and (for user metrics) authorized users. This is the input to metric mapping and position calculation. Read-only — it never contacts a publisher tool or changes a system; it operates only on the CSV you provide. HOW TO RUN powershell -ExecutionPolicy Bypass -File .\TIER1LIC-Discovery.ps1 ` -InventoryCsv .\inventory.csv -Publisher Oracle [-OutCsv .\deployment-Oracle.csv] Expected CSV columns (generic; map your own export to these headers): Host, Publisher, Product, Edition, Cores, CoreFactor, Hypervisor, Cluster, AuthorizedUsers, UserType EXPECTED RESULTS A filtered, normalized deployment inventory for the publisher printed to the console and, if -OutCsv is given, written to CSV for the position worksheet. .NOTES White-label SAM tooling — Asset Mgt. Read-only. Operates on inventory exports/CSV only; exact metric definitions are governed by your contracts and the publisher. #> [CmdletBinding()] param([Parameter(Mandatory)] [string] $InventoryCsv, [Parameter(Mandatory)] [ValidateSet('Oracle','IBM','SAP')] [string] $Publisher, [string] $OutCsv)

How to run: powershell -ExecutionPolicy Bypass -File .\TIER1LIC-Discovery.ps1 ` -InventoryCsv .\inventory.csv -Publisher Oracle [-OutCsv .\deployment-Oracle.csv] Expected CSV columns (generic; map your own export to these headers): Host, Publisher, Product, Edition, Cores, CoreFactor, Hypervisor, Cluster, AuthorizedUsers, UserType EXPECTED RESULTS A filtered, normalized deployment inventory for the publisher printed to the console and, if -OutCsv is given, written to CSV for the position worksheet. .NOTES White-label SAM tooling — Asset Mgt. Read-only. Operates on inventory exports/CSV only; exact metric definitions are governed by your contracts and the publisher. #> [CmdletBinding()] param([Parameter(Mandatory)] [string] $InventoryCsv, [Parameter(Mandatory)] [ValidateSet('Oracle','IBM','SAP')] [string] $Publisher, [string] $OutCsv)

TIER1LIC-Remediation.ps1

Purpose: Reads the exposure register and the mapped deployment and produces a remediation work list for products in shortfall: candidate uninstalls of unused installs, consolidation of processor-counted software onto fewer hosts, and removal of unentitled options/users. It is SAFE BY DEFAULT — it runs in -WhatIf mode and only proposes actions; it never uninstalls, moves, or changes any system. Approved actions must be performed through your normal change process. Re-run Validation and Report after each change. HOW TO RUN Preview (default, changes nothing): powershell -ExecutionPolicy Bypass -File .\TIER1LIC-Remediation.ps1 ` -RegisterCsv .\exposure-register-Oracle.csv -MappingCsv .\mapping-Oracle.csv ` -Publisher Oracle Emit the approved work list to CSV for the change process: ... -OutCsv .\remediation-worklist-Oracle.csv (There is no execute mode: this tool proposes only. Pass -Execute to acknowledge it is a no-op and confirm the proposal-only design.) EXPECTED RESULTS A proposed, prioritized work list of gap-closure candidates. No system is modified. .NOTES White-label SAM tooling — Asset Mgt. Proposal-only; default -WhatIf. Acting on a proposal is a business decision governed by your contracts and change process. #> [CmdletBinding(SupportsShouldProcess, ConfirmImpact='High')] param([Parameter(Mandatory)] [string] $RegisterCsv, [Parameter(Mandatory)] [string] $MappingCsv, [Parameter(Mandatory)] [ValidateSet('Oracle','IBM','SAP')] [string] $Publisher,

[string] $OutCsv, [switch] $WhatIf = $true, # default ON: safe preview [switch] $Execute # acknowledged no-op; this tool never changes a system)

How to run: Preview (default, changes nothing): powershell -ExecutionPolicy Bypass -File .\TIER1LIC-Remediation.ps1 `-RegisterCsv .\exposure-register-Oracle.csv -MappingCsv .\mapping-Oracle.csv` -Publisher Oracle Emit the approved work list to CSV for the change process: ... -OutCsv .\remediation-worklist-Oracle.csv (There is no execute mode: this tool proposes only. Pass -Execute to acknowledge it is a no-op and confirm the proposal-only design.) EXPECTED RESULTS A proposed, prioritized work list of gap-closure candidates. No system is modified. .NOTES White-label SAM tooling — Asset Mgt. Proposal-only; default -WhatIf. Acting on a proposal is a business decision governed by your contracts and change process. #> [CmdletBinding(SupportsShouldProcess, ConfirmImpact='High')] param([Parameter(Mandatory)] [string] $RegisterCsv, [Parameter(Mandatory)] [string] $MappingCsv, [Parameter(Mandatory)] [ValidateSet('Oracle','IBM','SAP')] [string] $Publisher, [string] $OutCsv, [switch] $WhatIf = $true, # default ON: safe preview [switch] $Execute # acknowledged no-op; this tool never changes a system)

TIER1LIC-Report.ps1

Purpose: Aggregates the mapped metric quantities per product and compares them to entitlements to produce the per-publisher licensing position: consumption required by the metric versus entitlement held, the resulting position (shortfall or headroom), an exposure quantity, and a severity. Output is the per-publisher exposure register. Read-only — it calculates from the files you supply and changes nothing. HOW TO RUN powershell -ExecutionPolicy Bypass -File .\TIER1LIC-Report.ps1 ` -MappingCsv .\mapping-Oracle.csv -EntitlementCsv .\entitlements-Oracle.csv ` -Publisher Oracle [-OutCsv .\exposure-register-Oracle.csv] MappingCsv columns (from Validation): Product, Metric, MetricQuantity, Unit, Flag EntitlementCsv columns: Product, Metric, Entitled (Entitled = licenses/PVU/users held) EXPECTED RESULTS A per-product table of consumption, entitlement, position, exposure, and severity, plus the exposure register CSV. Flagged (unmapped) rows are reported as unknown exposure, not zero. .NOTES White-label SAM tooling — Asset Mgt. Read-only. Entitlement quantities and exact metric definitions are governed by your contracts and the publisher; confirm there. #> [CmdletBinding()] param([Parameter(Mandatory)] [string] $MappingCsv, [Parameter(Mandatory)] [string] $EntitlementCsv, [Parameter(Mandatory)] [ValidateSet('Oracle','IBM','SAP')] [string] $Publisher, [string] $OutCsv)

How to run: powershell -ExecutionPolicy Bypass -File .\TIER1LIC-Report.ps1 `-MappingCsv .\mapping-Oracle.csv -EntitlementCsv .\entitlements-Oracle.csv` -Publisher Oracle [-OutCsv .\exposure-register-Oracle.csv] MappingCsv columns (from Validation): Product, Metric, MetricQuantity, Unit, Flag EntitlementCsv columns: Product, Metric, Entitled (Entitled = licenses/PVU/users held) EXPECTED RESULTS A per-product table of consumption, entitlement, position, exposure, and severity, plus the exposure register CSV. Flagged (unmapped) rows are reported as unknown exposure, not zero. .NOTES White-label SAM tooling — Asset Mgt. Read-only. Entitlement quantities and exact metric definitions are governed by your contracts and the publisher; confirm there. #> [CmdletBinding()] param([Parameter(Mandatory)] [string] $MappingCsv, [Parameter(Mandatory)] [string] $EntitlementCsv, [Parameter(Mandatory)] [ValidateSet('Oracle','IBM','SAP')] [string] $Publisher, [string] $OutCsv)

TIER1LIC-Set-ScopeTargeting.ps1

Purpose: Derives the set of hosts and users in scope for one tier-1 publisher from the inventory export, so the rest of the process (discovery, mapping, position, remediation) operates on a well-defined, repeatable

scope. Produces the publisher-scope group artifact. By default it only WRITES A LOCAL SCOPE FILE (it does not touch any directory or group system); pass -GroupExportPath to also emit a membership CSV your group/CMDB tooling can import. HOW TO RUN powershell -ExecutionPolicy Bypass -File .\TIER1LIC-Set-ScopeTargeting.ps1 ` -InventoryCsv .\inventory.csv -Publisher Oracle ` [-ScopeFile .\scope-Oracle.json] [-GroupExportPath .\scope-group-Oracle.csv] Inventory columns used: Host, Publisher, Product, AuthorizedUsers EXPECTED RESULTS The in-scope host list (and named users where present) for the publisher, saved as the scope group artifact; optionally a membership CSV for import into your group tooling. .NOTES White-label SAM tooling — Asset Mgt. Writes scope artifacts only; performs no changes against any directory, group, or device. Scope decisions are governed by your contracts and topology. #> [CmdletBinding(SupportsShouldProcess)] param([Parameter(Mandatory)] [string] $InventoryCsv, [Parameter(Mandatory)] [ValidateSet('Oracle','IBM','SAP')] [string] $Publisher, [string] $ScopeFile, [string] $GroupExportPath)

How to run: powershell -ExecutionPolicy Bypass -File .\TIER1LIC-Set-ScopeTargeting.ps1 `-InventoryCsv .\inventory.csv -Publisher Oracle` [-ScopeFile .\scope-Oracle.json] [-GroupExportPath .\scope-group-Oracle.csv] Inventory columns used: Host, Publisher, Product, AuthorizedUsers EXPECTED RESULTS The in-scope host list (and named users where present) for the publisher, saved as the scope group artifact; optionally a membership CSV for import into your group tooling. .NOTES White-label SAM tooling — Asset Mgt. Writes scope artifacts only; performs no changes against any directory, group, or device. Scope decisions are governed by your contracts and topology. #> [CmdletBinding(SupportsShouldProcess)] param([Parameter(Mandatory)] [string] $InventoryCsv, [Parameter(Mandatory)] [ValidateSet('Oracle','IBM','SAP')] [string] $Publisher, [string] $ScopeFile, [string] $GroupExportPath)

TIER1LIC-Validation.ps1

Purpose: For one tier-1 publisher, applies the recorded metric to each deployment row and produces the deployment-to-metric mapping: a metric quantity per install (processor cores adjusted by core factor, value units per core, named users by type, or document volume for digital access). Rows that cannot be mapped (missing metric, missing cores/core factor, sub-capacity claimed without a prerequisite) are flagged — an unmapped row is unknown exposure. Read-only. HOW TO RUN powershell -ExecutionPolicy Bypass -File .\TIER1LIC-Validation.ps1 ` -InventoryCsv .\deployment-Oracle.csv -Publisher Oracle [-OutCsv .\mapping-Oracle.csv] CSV columns (from Discovery): Host, Publisher, Product, Edition, Cores, CoreFactor, Hypervisor, Cluster, AuthorizedUsers, UserType [, Metric, SubCapacityProven, ValueUnitPerCore] Metric is the governing metric per product from your contract; supply it in the CSV or rely on the generic inference below (which you MUST confirm against the contract). EXPECTED RESULTS Per row: a computed metric quantity, or a clear flag. A summary of mapped vs flagged rows. .NOTES White-label SAM tooling — Asset Mgt. Read-only. Counting rules summarized at a general level; exact core factors, user categories, and rates are governed by your contracts and the publisher. #> [CmdletBinding()] param([Parameter(Mandatory)] [string] $InventoryCsv, [Parameter(Mandatory)] [ValidateSet('Oracle','IBM','SAP')] [string] $Publisher, [string] $OutCsv)

How to run: powershell -ExecutionPolicy Bypass -File .\TIER1LIC-Validation.ps1 ` -InventoryCsv .\deployment-Oracle.csv -Publisher Oracle [-OutCsv .\mapping-Oracle.csv] CSV columns (from Discovery): Host, Publisher, Product, Edition, Cores, CoreFactor, Hypervisor, Cluster, AuthorizedUsers, UserType [, Metric, SubCapacityProven, ValueUnitPerCore] Metric is the governing metric per product from your contract; supply it in the CSV or rely on the generic inference below (which you MUST confirm against the contract). EXPECTED RESULTS Per row: a computed metric quantity, or a clear flag. A summary of mapped vs flagged rows. .NOTES White-label SAM tooling — Asset Mgt. Read-only.

Counting rules summarized at a general level; exact core factors, user categories, and rates are governed by your contracts and the publisher. #> [CmdletBinding()] param([Parameter(Mandatory)] [string] $InventoryCsv, [Parameter(Mandatory)] [ValidateSet('Oracle','IBM','SAP')] [string] $Publisher, [string] $OutCsv)